Atividade 3: Controle de Saída Digital com PWM no ESP32 no Simulador Wokwi

Discente: Andreza Costa dos Santos

Objetivo:

Desenvolver uma dispositivo para controlar 4 LEDs e um buzzer piezoelétrico utilizando o PWM (driver LEDC) do ESP32.

Esta atividade será realizada no simulador Wokwi e deve utilizar a ESP-IDF como framework de desenvolvimento.

Material Necessário:

- Conta no Wokwi (https://wokwi.com/)
- Conhecimento básico de ESP-IDF
- O ESP32 como microcontrolador
- 4 LEDs
- 4 resistores limitadores de corrente
- 1 buzzer piezoelétrico

Passos para a Atividade:

1. **Diagrama em Blocos**

   Elabore o diagrama em blocos representando:

   - EESP32-S3: microcontrolador responsável pelo acionamento das saídas PWM.
   - LED1, LED2, LED3 e LED4: atuadores luminosos controlados por PWM. ● Buzzer: atuador sonoro controlado por PWM.

2. **Esquemático:**

   Desenhe o circuito esquemático, incluindo:

   - 4 LEDs conectados a GPIOs configuradas como saída PWM, cada um em série com seu resistor.
   - Buzzer conectado a outra GPIO configurada como saída PWM.
   - Alimentação de 3,3 V fornecida pelo ESP32.

3. **Configuração do Ambiente de Simulação:**

   Crie o circuito no Wokwi:

   - Acesse o simulador Wokwi e crie um novo projeto.
   - Adicione o ESP32-S3 ao projeto.

- Adicione 4 LEDs e seus resistores.
- Adicione o buzzer piezoelétrico.
- Faça as conexões nas GPIOs configuradas para PWM.

4. Desenvolvimento do Código:

Implemente em C com ESP-IDF utilizando o driver LEDC:

- Configure 5 canais PWM (LEDC), um para cada LED e um para o buzzer.
- Configure frequência e resolução:
  - LEDs → 1 kHz (brilho suave).
  - Buzzer → frequência variável (500 Hz a 2 kHz).
- Crie funções para:
  - Alterar duty cycle dos LEDs (0% a 100%).
  - Alterar frequência do buzzer (sons diferentes).

Lógica do Programa:

- **Fase 1**: Fading Sincronizado dos LEDs
  - Todos os LEDs variam o brilho de 0% → 100% → 0% em ciclos.
- **Fase 2**: Fading Sequencial dos LEDs
  - LED1 faz o ciclo → LED2 → LED3 → LED4, depois retorna.
- **Fase 3**: Teste Sonoro com o Buzzer
  - O buzzer emite tons que variam de 500 Hz a 2000 Hz.
  - Frequência sobe gradualmente (500 → 2000 Hz) e depois desce.

Observações Importantes

- Todos os LEDs devem iniciar apagados.
- LEDs iniciam apagados (duty = 0%).
- Frequência inicial do buzzer: 1000 Hz.
- O tempo de atualização deve ser configurável via #define (por exemplo: #define DELAY_MS 500).
- A lógica deve estar organizada em funções para facilitar a manutenção do código.
- Utilizar o driver LEDC.

Desenvolvimento

Detalhamento e pesquisa:

1. O Coração do Projeto: ESP32-S3 e PWM

O objetivo central é usar o periférico **LEDC** para controlar atuadores luminosos e sonoros. Diferente de uma saída digital comum (ligado/desligado), o **PWM (Pulse Width Modulation)** permite simular valores analógicos variando a largura do pulso.

- **Para os LEDs:** O PWM controla a intensidade do brilho (Duty Cycle).
- **Para o Buzzer:** O PWM controla a frequência da oscilação, o que determina a nota musical ou o tom emitido.

---

2. Lista de Componentes e Conexões

Para a simulação no Wokwi, precisaremos organizar os seguintes itens:

Atuadores e Proteção

- **1x Microcontrolador ESP32-S3**.
- **4x LEDs:** Serão os nossos indicadores luminosos.
- **4x Resistores:** Essenciais para limitar a corrente e não "queimar" os LEDs na vida real (no simulador, evitam avisos de sobrecorrente).
- **1x Buzzer Piezoelétrico:** Para a saída sonora.

Diagrama de Conexões (Sugestão)

No Wokwi, devemos conectar os componentes da seguinte forma:

Componente	Conexão Positiva (Anodo)	Conexão Negativa (Catodo/GND)
LED 1	GPIO 1 (via Resistor)	GND
LED 2	GPIO 2 (via Resistor)	GND
LED 3	GPIO 42 (via Resistor)	GND
LED 4	GPIO 41 (via Resistor)	GND
Buzzer	GPIO 5	GND

---

3. Conceito do Driver LEDC (ESP-IDF)

Para configurar o código seguindo as boas práticas do framework, precisamos entender que o LEDC é dividido em dois blocos principais:

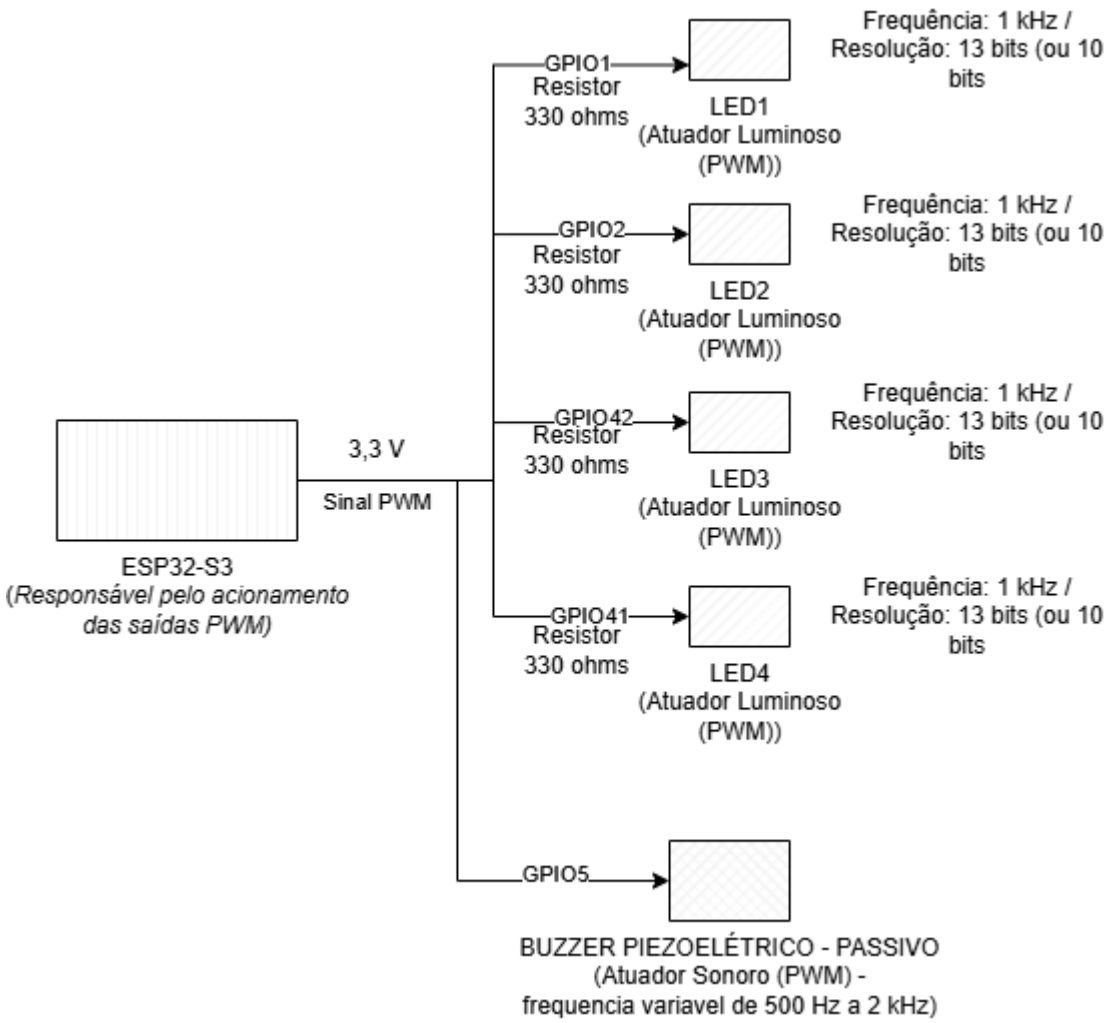
1. **Timer Configuration:** Define a "batida" do PWM. Aqui configuramos a **frequência** (ex: 1 kHz para os LEDs) e a **resolução** (quantos bits definem a precisão do brilho).

2. **Channel Configuration:** Associa o Timer a uma **GPIO** específica e define o **Duty Cycle** inicial.

Dica de Ouro: Como o buzzer precisa variar a frequência de 500 Hz a 2000 Hz , ele precisará de um **Timer exclusivo** separado do Timer dos LEDs, que ficarão fixos em 1 kHz.

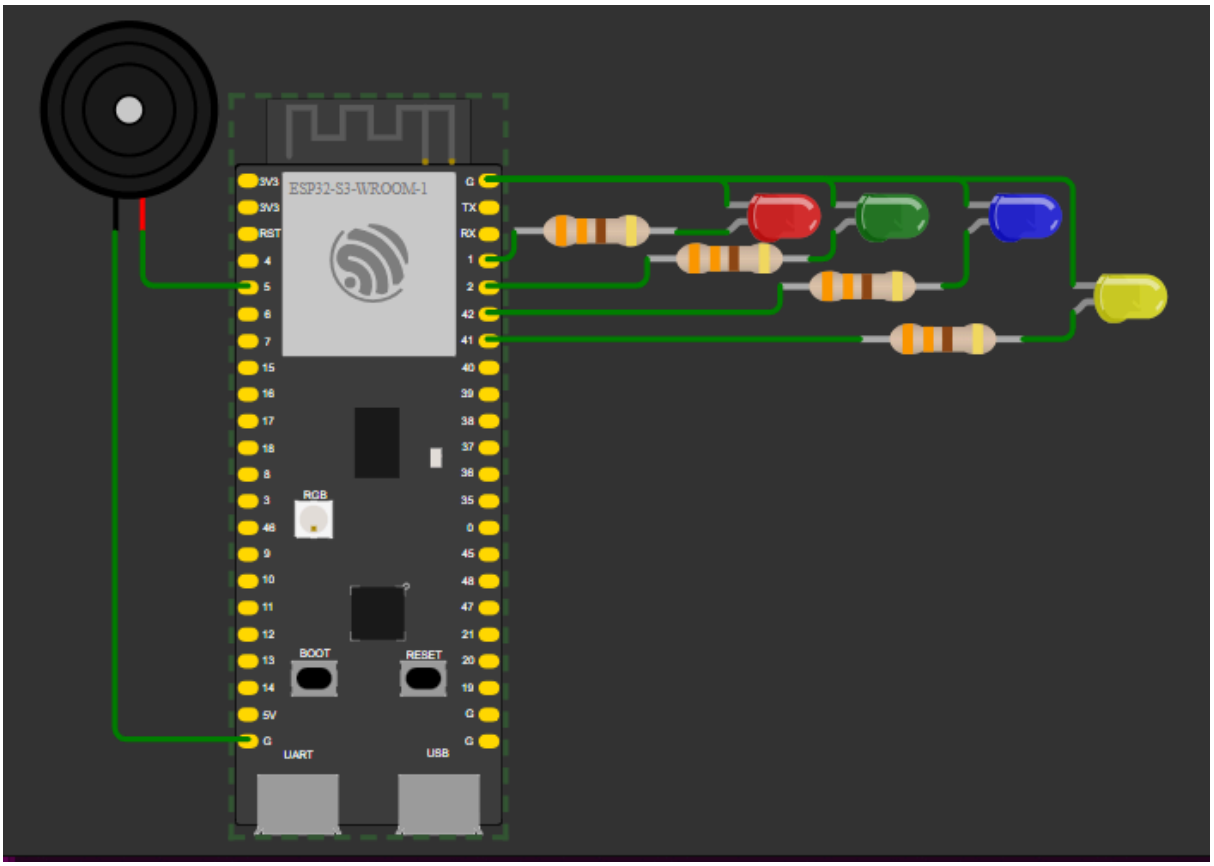
—-

PASSO 1: DIAGRAMA DE BLOCOS

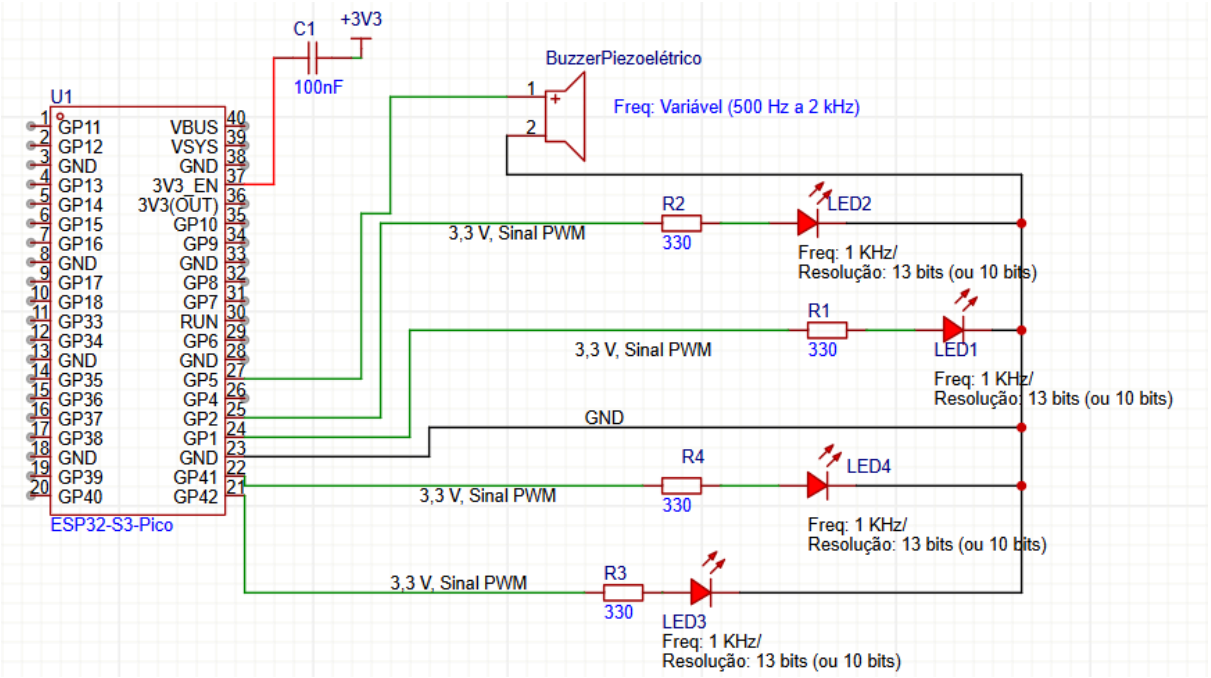


—

PASSO 2: DIAGRAMA ESQUEMÁTICO



OBS: Para a atividade completa foi considerada a utilização do modelo de ESP acima disponibilizado no simulador, mas por questões de indisponibilidades do modelo via software para diagramar, foi utilizado o modelo ESP32-S3-PICO no EasyEDA (online), que não disponibiliza o modelo acima. **Segue o diagrama esquemático:**



—-

PASSOS 3 E 4:

1) CÓDIGO:

```
#include <stdio.h>
```

```
#include "freertos/FreeRTOS.h"

#include "freertos/task.h"

#include "driver/ledc.h"



// --- CONFIGURAÇÕES VIA #DEFINE (Requisitos do Roteiro) ---

#define DELAY_FADE_MS      15     // Velocidade do efeito de fade

#define LEDC_RESOLUTION    LEDC_TIMER_13_BIT

#define LEDC_MAX_DUTY      8191   // 2^13 - 1
```

```c
#define LED_FREQ          1000  // 1 -kHz para LEDs
#define BUZZER_GPIO       5     // Pino do Buzzer
#define BUZZER_INIT_FREQ  1000  // Frequência inicial

// GPIOs dos LEDs conforme seu esquemático
const int led_pins[] = {1, 2, 42, 41};

// --- PROTÓTIPOS DAS FUNÇÕES ---
void configurar_pwm();
void set_led_duty(int canal, int duty);
void set_buzzer_freq(int freq);
void fase1_sincronizado();
void fase2_sequencial();
void fase3_buzzer();

// --- FUNÇÃO PRINCIPAL ---
void app_main(void) {
    // Inicializa o hardware
    configurar_pwm();

    while (1) {
        printf(">>> FASE 1: Fading Sincronizado\n");
```

```
        fase1_sincronizado();
        vTaskDelay(pdMS_TO_TICKS(1000));

        printf(">>> FASE 2: Fading Sequencial\n");
        fase2_sequencial();
        vTaskDelay(pdMS_TO_TICKS(1000));

        printf(">>> FASE 3: Teste do Buzzer\n");
        fase3_buzzer();
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

// --- IMPLEMENTAÇÃO DAS FUNÇÕES ---

void configurar_pwm() {
    // 1. Configuração do Timer para LEDs (1kHz)
    ledc_timer_config_t led_timer = {
        .speed_mode = LEDC_LOW_SPEED_MODE,
        .duty_resolution = LEDC_RESOLUTION,
        .timer_num = LEDC_TIMER_0,
        .freq_hz = LED_FREQ,
```

```
        .clk_cfg = LEDC_AUTO_CLK
    };
    ledc_timer_config(&led_timer);

    // 2. Configuração do Timer para Buzzer
    ledc_timer_config_t buzz_timer = {
        .speed_mode = LEDC_LOW_SPEED_MODE,
        .duty_resolution = LEDC_RESOLUTION,
        .timer_num = LEDC_TIMER_1,
        .freq_hz = BUZZER_INIT_FREQ,
        .clk_cfg = LEDC_AUTO_CLK
    };
    ledc_timer_config(&buzz_timer);

    // 3. Configuração dos Canais dos LEDs
    for (int i = 0; i < 4; i++) {
        ledc_channel_config_t ch = {
            .gpio_num = led_pins[i],
            .speed_mode = LEDC_LOW_SPEED_MODE,
            .channel = (ledc_channel_t)i,
            .timer_sel = LEDC_TIMER_0,
            .duty = 0,            // Define duty inicial como 0
```

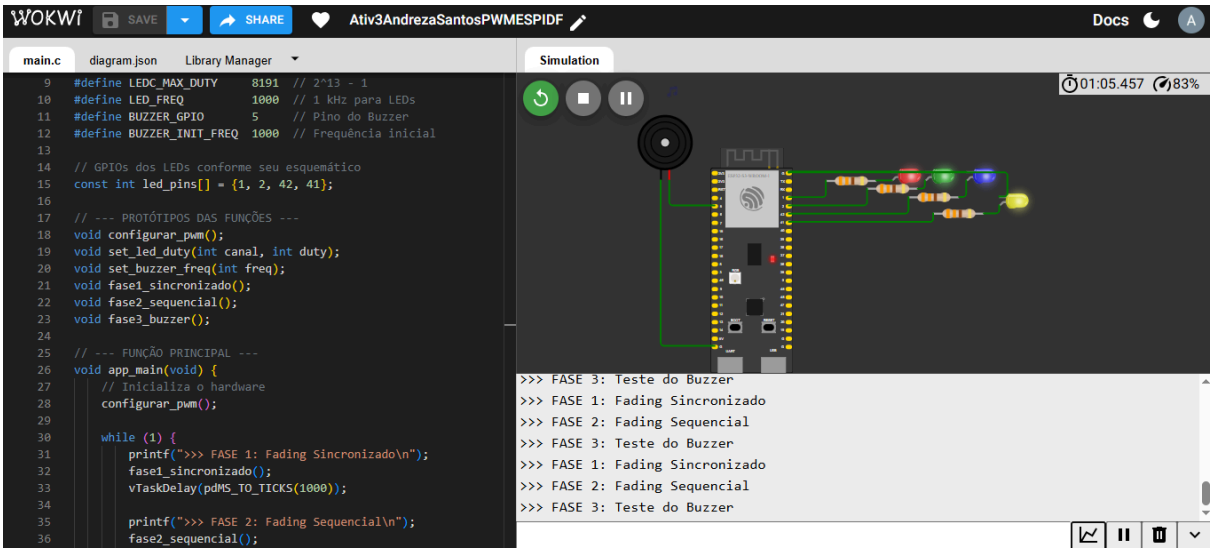
```
            .hpoint = 0

        };

        ledc_channel_config(&ch);



        // Comando extra de segurança para garantir que iniciem
APAGADOS

        ledc_set_duty(LEDC_LOW_SPEED_MODE, (ledc_channel_t)i, 0);

        ledc_update_duty(LEDC_LOW_SPEED_MODE, (ledc_channel_t)i);

    }



    // 4. Configuração do Canal do Buzzer

    ledc_channel_config_t b_ch = {

        .gpio_num = BUZZER_GPIO,

        .speed_mode = LEDC_LOW_SPEED_MODE,

        .channel = LEDC_CHANNEL_4,

        .timer_sel = LEDC_TIMER_1,

        .duty = 0,                // Silêncio inicial

        .hpoint = 0

    };

    ledc_channel_config(&b_ch);

    ledc_set_duty(LEDC_LOW_SPEED_MODE, LEDC_CHANNEL_4, 0);

    ledc_update_duty(LEDC_LOW_SPEED_MODE, LEDC_CHANNEL_4);

}
```

```
// Função para alterar brilho do LED
void set_led_duty(int canal, int duty) {
    ledc_set_duty(LEDC_LOW_SPEED_MODE, (ledc_channel_t)canal,
duty);
    ledc_update_duty(LEDC_LOW_SPEED_MODE, (ledc_channel_t)canal);
}

// Função para alterar frequência do Buzzer
void set_buzzer_freq(int freq) {
    ledc_set_freq(LEDC_LOW_SPEED_MODE, LEDC_TIMER_1, freq);
    // Duty cycle em 50% (aprox 4095 para 13 bits) para gerar onda
sonora
    ledc_set_duty(LEDC_LOW_SPEED_MODE, LEDC_CHANNEL_4, 4095);
    ledc_update_duty(LEDC_LOW_SPEED_MODE, LEDC_CHANNEL_4);
}

// FASE 1: Todos os LEDs juntos
void fase1_sincronizado() {
    for (int d = 0; d <= LEDC_MAX_DUTY; d += 200) {
        for (int i = 0; i < 4; i++) set_led_duty(i, d);
        vTaskDelay(pdMS_TO_TICKS(DELAY_FADE_MS));
    }
```

```
    for (int d = LEDC_MAX_DUTY; d >= 0; d -= 200) {
        for (int i = 0; i < 4; i++) set_led_duty(i, d);
        vTaskDelay(pdMS_TO_TICKS(DELAY_FADE_MS));
    }
}

// FASE 2: Um LED de cada vez
void fase2_sequencial() {
    for (int i = 0; i < 4; i++) {
        for (int d = 0; d <= LEDC_MAX_DUTY; d += 400) {
            set_led_duty(i, d);
            vTaskDelay(pdMS_TO_TICKS(DELAY_FADE_MS));
        }
        for (int d = LEDC_MAX_DUTY; d >= 0; d -= 400) {
            set_led_duty(i, d);
            vTaskDelay(pdMS_TO_TICKS(DELAY_FADE_MS));
        }
    }
}

// FASE 3: Buzzer subindo e descendo frequência
void fase3_buzzer() {
```

```
    // Sobe: 500 Hz -> 2000 Hz

    for (int f = 500; f <= 2000; f += 50) {

        set_buzzer_freq(f);

        vTaskDelay(pdMS_TO_TICKS(50));

    }

    // Desce: 2000 Hz -> 500 Hz

    for (int f = 2000; f >= 500; f -= 50) {

        set_buzzer_freq(f);

        vTaskDelay(pdMS_TO_TICKS(50));

    }

    // Silencia o buzzer ao final da fase

    ledc_set_duty(LEDC_LOW_SPEED_MODE, LEDC_CHANNEL_4, 0);

    ledc_update_duty(LEDC_LOW_SPEED_MODE, LEDC_CHANNEL_4);

}
```

2) PRINT SIMULAÇÃO:



3) LINK DE ACESSO A SIMULAÇÃO:

https://wokwi.com/projects/459850418666820609